

Module: Ubuntu 101 - Fundamentals

Session 1: Introduction to Ubuntu

Explanation

Understanding the basics of Ubuntu, its open-source nature, and key components.

1. Understanding the Basics of Ubuntu

Ubuntu is a popular Linux distribution based on Debian and developed by Canonical. It is widely used for desktop computers, servers, cloud environments, and development platforms.

Understanding Ubuntu involves learning:

- File system structure
- Command Line Interface (CLI)
- User accounts and permissions
- Process management
- Software installation and updates

Familiarity with basic commands such as `ls`, `cd`, `cp`, and `grep` is essential. Ubuntu uses the **APT (Advanced Package Tool)** package management system for installing and updating software.

2. Open Source Nature

Ubuntu is open-source software, which means its source code is freely available for anyone to view, modify, and distribute. This open-source model encourages collaboration among developers, improves security through transparency, and allows organizations and individuals to customize the operating system to suit their requirements.

3. Key Components of Ubuntu

Kernel

The core of the operating system that manages hardware resources and communication between software and hardware.

Shell

The interface through which users interact with Ubuntu, primarily through the command line.

File System

Ubuntu uses a hierarchical file system structure with important directories such as:

- `/home` – User files and directories
- `/etc` – System configuration files
- `/var` – Logs and variable data
- `/usr` – Applications and system utilities

Utilities

A collection of command-line tools used for file management, process monitoring, networking, and system administration.

Users and Permissions

Ubuntu is a multi-user operating system that uses permissions to control access to files and resources.

Package Management

Ubuntu uses **APT** for software installation, updates, and removal.

Networking

Ubuntu includes built-in networking tools and supports various network protocols for communication across local and remote networks.

Note

These components form the foundation of Ubuntu, providing stability, security, and flexibility for desktop and server environments.

Session 2: Navigating the Terminal

Explanation

Introduction to the command line and basic navigation commands.

1. pwd (Print Working Directory)

Displays the current working directory.

Example

```
pwd
```

2. ls (List)

Lists files and directories.

Example

```
ls
```

Lists files and directories in the current directory.

```
ls -R
```

Lists files within subdirectories.

```
ls -a
```

Lists hidden files.

```
ls -la
```

Displays detailed file information.

3. cd (Change Directory)

Changes the current directory.

Examples

Navigate to a specific directory:	<code>cd directory_name</code>
Navigate to the Home directory:	<code>cd ~</code>
Move to the root directory:	<code>cd /</code>
Move one level up:	<code>cd ..</code>

Note

These commands are essential for navigating the Ubuntu file system efficiently.

Session 3: File Operations

Explanation

Basic file manipulation and directory management in Ubuntu.

File Commands

1. touch

Creates an empty file.

Syntax

```
touch file_name
```

Examples

```
touch file.txt  
touch /home/juan/Documents/file.txt
```

2. cp (Copy)

Copies files or directories.

Syntax

```
cp source_file destination
```

Example

```
cp file.txt /home/juan/Documents/
```

3. mv (Move)

Moves or renames files and directories.

Syntax

```
mv source destination
```

Example

```
mv file.txt /home/juan/Documents/
```

Rename a file:

```
mv file.txt report.txt
```

4. rm (Remove)

Deletes files.

Syntax

```
rm file_name
```

Example

```
rm unwanted_file.txt
```

5. ln (Link)

Creates hard or symbolic links.

Syntax

```
ln source link_name  
ln -s source link_name
```

Examples

```
ln file.txt file_link  
ln -s file.txt symlink_file
```

Working with Directories

mkdir (Make Directory)

Creates a directory.

Syntax

```
mkdir directory_name
```

Example

```
mkdir new_directory
```

rmdir (Remove Directory)

Removes an empty directory.

Syntax

```
rmdir directory_name
```

Example

```
rmdir empty_directory
```

Note

These commands are fundamental for file and directory management. Use `rm` and `mv` carefully, as their actions may be difficult to reverse.

Session 4: Text Editing with Nano

Explanation

Nano is a simple and user-friendly text editor included by default in Ubuntu.

1. Opening or Creating a File

Syntax

```
nano filename.txt
```

Example

```
nano notes.txt
```

2. Saving Changes

- Press **Ctrl + O**
 - Press **Enter** to confirm the filename
-

3. Exiting Nano

- Press **Ctrl + X**
 - If changes were made, Nano will prompt to save them
-

4. Navigating

Use the arrow keys to move through the file.

5. Cut, Copy, and Paste

- **Ctrl + K** : Cut current line
 - **Alt + ^** : Copy current line
 - **Ctrl + U** : Paste text
-
-

6. Find and Replace

- **Ctrl + W** : Search for text
- **Ctrl + ** : Search and replace

Note

Nano is ideal for beginners because it displays available shortcuts at the bottom of the screen.

Session 5: User Management

Explanation

Managing user accounts and permissions in Ubuntu.

1. Creating Users

Use the `adduser` command to create a new user.

Syntax

```
Shell
1
sudo adduser username
Show more lines
```

Example

```
Shell
1
sudo adduser pedro
Show more lines
```

2. Deleting Users

Remove a user and the user's home directory.

Syntax

```
Shell
1
sudo deluser --remove-home username
Show more lines
```

Example

```
Shell
1
sudo deluser --remove-home pedro
Show more lines
```

3. Managing Passwords

Change a user's password.

Syntax

```
Shell
1
sudo passwd username
Show more lines
```

Example

```
Shell
1
sudo passwd pedro
Show more lines
```

4. Viewing User Information

Display user and group information.

Syntax

```
Shell
1
id username
Show more lines
```

Example

```
Shell
1
id pedro
Show more lines
```

5. Granting Administrative Privileges

Ubuntu uses the **sudo** group for administrative access.

Syntax

```
sudo usermod -aG sudo username
```

Example

```
sudo usermod -aG sudo pedro
```

Verify group membership:

```
groups pedro
```

Note

Replace "username" with the actual username when using these commands.

Session 6: Permissions and Ownership

Explanation

Understanding file permissions (`chmod`) and ownership (`chown`).

File Permissions

Example:

-rw-rw-r--

Permission breakdown:

- First character (-) indicates a regular file.
- Owner has read and write permissions (**rw**-).
- Group has read and write permissions (**rw**-).
- Others have read-only permission (**r**--).

Permission Values

Permission	Value
Read (r)	4
Write (w)	2
Execute (x)	1

Example

Desired permissions:

- Owner: Read + Write = 6
- Group: Read = 4
- Others: Read = 4

Result:

644

Change Permissions

Syntax

`sudo chmod ### filename`

Example

```
sudo chmod 644 example.txt
```

Result:

```
-rw-r--r--
```

File Ownership

Example:

```
-rw-r--r-- 1 user1 group1 1024 Mar 10 12:34 example.txt
```

Change Ownership

Syntax

```
sudo chown newuser:newgroup filename
```

Example

```
sudo chown user2:group2 example.txt
```

Result:

```
-rw-r--r-- 1 user2 group2 1024 Mar 10 12:34 example.txt
```

Note

Understanding permissions and ownership is critical for maintaining system security and controlling file access.

Session 7: Introduction to Package Management

Explanation

Ubuntu uses **APT (Advanced Package Tool)** to manage software packages.

1. Update Package Information

```
sudo apt update
```

Downloads the latest package information from configured repositories.

2. Upgrade Installed Packages

```
sudo apt upgrade
```

Updates installed software to the latest available versions.

3. Install a Package

Syntax

```
sudo apt install package-name
```

Example

```
sudo apt install htop
```

4. Remove a Package

Syntax

```
sudo apt remove package-name
```

Example

```
sudo apt remove htop
```

5. Remove a Package and Configuration Files

Syntax

```
sudo apt purge package-name
```

Example

```
sudo apt purge htop
```

6. Search for Packages

Syntax

```
apt search package-name
```

Example

```
apt search nginx
```

7. View Package Information

Syntax

```
apt show package-name
```

Example

```
apt show nginx
```

Note

APT automatically handles package dependencies, making software installation and maintenance simpler and more reliable.